

Instituto Tecnológico Superior del Sur de Guanajuato
“Tecnología y calidad para la vida”



Programación Móvil I

Realizado por:

Fujarte Herrera Ambar Paola

Lopez Mora Alexander

Maldonado Nava Janetzy

Semestre: Séptimo

Carrera: Ingeniería en Sistemas Computacionales

Profesora: Gustavo Ivan Vega Olvera

Uriangato, Gto.

04 de Octubre del 2026

1. Identificación y Descripción de la Problemática

En Acámbaro, el pan tradicional (el pan grande de huevo, las acambaritas, los picones) es uno de los productos más importantes tanto para la gente local como para los turistas que visitan la región. Sin embargo, las panaderías de aquí siguen trabajando de manera muy tradicional y tienen problemas cotidianos que se pueden resolver con una app:

La gente no sabe cuándo sale el pan caliente: Muchas veces vas a la panadería y el pan que buscas todavía no sale del horno o ya se acabó la tanda. No hay forma de saber los horarios de horneado sin ir a dar la vuelta físicamente.

Descontrol con los encargos grandes: Cuando alguien encarga varias piezas para un evento, viaje o fiesta familiar, casi siempre se anota en una libreta de mostrador o se pide por mensaje. Esto provoca que se confundan los pedidos, no se tenga listo el empaque a tiempo o se generen filas innecesarias.

Mala señal o falta de internet en algunos puntos: En varias zonas o dentro de los mismos locales la señal de datos falla. Si la app dependiera al 100% de internet, se trabaría a cada rato.

¿Cuál es la solución?

Crear PanTradición Acámbaro, una aplicación nativa en Android hecha con Jetpack Compose. La app le permitirá a los clientes armar su encargo de pan, elegir el tipo de empaque, poner la hora exacta en la que pasarán a recogerlo y ver su historial de pedidos.

La característica clave es que funcionará con una arquitectura Offline-First: todo se guardará primero en la base de datos local del teléfono (Room) para que la app nunca se quede congelada si no hay señal, y cuando haya internet, se sincronizará con un servidor en la nube.

Requerimientos Funcionales (RF)

- RF-01 (Catálogo de piezas): La aplicación debe mostrar la lista de panes tradicionales disponibles (pan de huevo, picones, acambaritas, semita) con su nombre, descripción artesanal y precio unitario.
- RF-02 (Selección múltiple): El usuario debe poder seleccionar una o varias piezas de pan del catálogo utilizando casillas de verificación (Checks) y tarjetas interactivas.
- RF-03 (Tipo de empaque): La aplicación debe permitir elegir una sola opción de empaque (Caja tradicional de cartón o Bolsa ecológica de papel) mediante botones de selección única (Radios).
- RF-04 (Servicio de rebanado): El usuario debe poder marcar mediante un Checkbox si requiere que el pan sea rebanado previamente para eventos, reuniones o viajes.
- RF-05 (Programación de horario de recolección): La aplicación debe integrar un selector de hora interactivo (TimePicker) para que el cliente elija la hora exacta en la que pasará al mostrador por su pedido.
- RF-06 (Notas especiales para el panadero): La app debe contar con una caja de texto multilínea (TextField) para que el cliente escriba instrucciones específicas sobre su encargo (por ejemplo: *"apartar de la tanda caliente de la tarde"*).

- RF-07 (Validación de encargo): El sistema debe validar que el usuario haya seleccionado al menos una pieza de pan antes de habilitar o permitir la confirmación del pedido.
- RF-08 (Guardado local y modo offline): La aplicación debe guardar los pedidos confirmados en la base de datos interna del dispositivo sin necesidad de conexión activa a internet.
- RF-09 (Historial de encargos): El usuario debe poder consultar en una pantalla dedicada la lista de todos los pedidos registrados en el dispositivo con sus respectivos detalles y fecha.

Requerimientos No Funcionales (RNF)

- RNF-01 (Interfaz 100% nativa con Jetpack Compose): Toda la interfaz de usuario debe construirse con Jetpack Compose y componentes de Material Design 3, eliminando por completo el uso de layouts XML tradicionales.
- RNF-02 (Arquitectura MVVM): El proyecto debe separar estrictamente la vista de la lógica de negocio mediante ViewModels, exponiendo el estado con StateFlow y capturando acciones mediante eventos unidireccionales (UiEvent).
- RNF-03 (Patrón Repository): Toda operación de datos debe pasar por una capa de repositorio (BakeryRepository) que centralice el acceso a los orígenes locales y remotos.
- RNF-04 (Persistencia de datos con Room Database): Se debe utilizar la librería oficial Room (SQLite) como fuente local de la verdad para garantizar la operación offline.
- RNF-05 (Soporte para pantallas adaptativas): La interfaz debe adaptarse dinámicamente según el tamaño de pantalla (WindowWidthSizeClass), mostrando una sola columna en teléfonos y dos columnas paralelas en tablets o dispositivos en modo horizontal.
- RNF-06 (Internacionalización - i18n): La aplicación debe ofrecer soporte para múltiples idiomas (Español e Inglés) configurados a través de archivos de recursos (strings.xml), sin textos fijos en el código Kotlin.
- RNF-07 (Navegación centralizada): La transición entre pantallas debe gestionarse mediante el componente oficial NavHost de Jetpack Compose y una barra de navegación inferior (NavigationBar).
- RNF-08 (Preparación para servicios en la nube): La estructura del repositorio debe permitir la integración posterior de un cliente HTTP (Retrofit) para sincronizar las órdenes locales con una API Web en la nube.

3. Propuesta Técnica y Arquitectura

Capa UI (Compose): Son las pantallas (OrderScreen, HistoryScreen). Solo se encargan de pintar los componentes (botones, listas, textos) y detectar los toques del usuario. No calculan precios ni hacen consultas a bases de datos directamente.

ViewModel (OrderViewModel): Es el "cerebro" de la pantalla. Guarda variables como la hora seleccionada, el empaque elegido y los panes marcados. Si el usuario rota la pantalla o entra una llamada, el ViewModel evita que la información se pierda.

Repository (BakeryRepository): Es el intermediario. El ViewModel le dice "guárdame esta orden", y el repositorio se encarga de guardarla en la base de datos local y, cuando haya red, enviarla al servidor.

Base de Datos Local (Room): Guarda las tablas de panes y pedidos dentro del dispositivo. Si el celular se queda sin señal en la panadería, la app sigue abriendo y guardando pedidos sin problemas.

Servicios Web (API / Nube): Conexión con un backend (mediante Retrofit) para sincronizar pedidos y actualizar precios en tiempo real cuando se detecte conexión a internet.

4. Criterios de Aceptación Cumplidos en esta Fase

Elementos de interacción:

- Listas: LazyColumn para desplegar el catálogo de piezas y la lista del historial.
- Cajas de texto: OutlinedTextField para capturar notas del usuario.
- Radios: RadioButton para la selección única de empaque (caja o bolsa).
- Checks: Checkbox para seleccionar panes y marcar el servicio de rebanado.
- TimePicker: Diálogo interactivo de Material 3 para elegir la hora de recogida.
- Botones: Button y OutlinedButton con eventos de guardado y apertura de modales.
- Manejo de estado: Implementado mediante OrderViewModel con StateFlow y eventos bien definidos.
- Navegación: NavHost funcional que permite alternar entre la pantalla de Encargo y la de Historial mediante la barra inferior.
- Pantallas adaptativas: Detección de tamaño de ventana (WindowWidthSizeClass) para acomodar la UI en una sola columna en celulares o en dos columnas paralelas en tablets.
- Soporte de idiomas: Textos separados en values/strings.xml (Español) y values-en/strings.xml (Inglés).
- Base de datos local: Base de datos Room creada, con catálogo precargado y guardado real de pedidos.

5. Plan de Trabajo en Equipo (GitHub)

El proyecto se organizará en un repositorio de GitHub.

- Se utilizará la rama main para las versiones estables listas para entrega al profesor y develop para la integración del equipo.
- Cada integrante trabajará en ramas de características (feature/ui-compose, feature/room-database, feature/navigation).
- Se mantendrá un historial limpio mediante commits descriptivos para evidenciar el trabajo colaborativo de los integrantes.